

11.1.2 Sum of Digits using Recursion

A] Algorithm

Step 1: Start

Step 2: Read an integer from the user and store it in number

Step 3: Define a recursive function sum_of_digits_recursive(n)

Step 4: Inside the function:

Step 4a: If $n == 0$, return 0

Step 4b: Otherwise, return $(n \% 10) + \text{sum_of_digits_recursive}(n // 10)$

Step 5: Call sum_of_digits_recursive(number) and store the result in result

Step 6: Print result **Step 7:** End

B] code def

sum_of_digits_recursive(n):

 if $n == 0$:

 return 0

else:

 return $(n \% 10) + \text{sum_of_digits_recursive}(n // 10)$

number=int(input()) result =

sum_of_digits_recursive(number)

print(result)

C] output

The screenshot displays a coding environment with the following components:

- Problem Statement (Left Panel):** Titled "11.1.2. Sum of Digits using Recursion". It asks to write a recursive function `sum_of_digits_recursive(n)` that calculates the sum of the digits of a given non-negative integer n . It includes input/output formats, constraints ($0 \leq n \leq 10^9$), and sample test cases: Test case 1 (input 23, output 5) and Test case 2 (input 1234, output 10).
- Code Editor (Center):** Shows the implementation of the recursive function:

```
1 def sum_of_digits_recursive(n):
2     if n == 0:
3         return 0
4     else:
5         return (n % 10) + sum_of_digits_recursive(n // 10)
6
7
8 number = int(input())
9
10 result = sum_of_digits_recursive(number)
11
12 print(result)
13
14
```
- Execution Results (Bottom Right):** Displays performance metrics (Average time: 0.008 s, Maximum time: 0.013 s) and test case outcomes. It shows that 2 out of 2 shown test cases passed and 3 out of 3 hidden test cases passed. Details for Test case 1 show an expected output of 5 and an actual output of 5.

D] flowchart

